TECHNICAL NOTE

Native Windows Compilation and Deployment of OpenMC Monte Carlo Particle Transport Code

Sachin Shet
iamsachinshet@gmail.com
https://www.kinkedin.com/in/sac18

Abstract

OpenMC is an open-source, continuous-energy Monte Carlo particle transport code widely used in nuclear engineering research and reactor physics analysis. It is officially supported only on Linux/macOS, creating a significant barrier for Windows users. This technical note describes the fully native Windows executable (openmc.exe) of OpenMC v0.15.3, compiled using the GCC 15.2.0. The resulting binary requires no Windows Subsystem for Linux (WSL) or Docker runtime, integrates with the standard OpenMC Python API, and is distributed as a self-contained Windows installer. This binary supports the official HDF5 cross section data files. This work enables direct OpenMC usage on Windows workstations and classrooms without virtualisation overhead.

Keywords: *OpenMC, Monte Carlo, Windows, Radiation Transport, Native Build, HDF5*

1. Introduction

OpenMC [1,2] is a community-developed, open-source Monte Carlo code capable of simulating neutron and photon transport in complex three-dimensional geometries. It has been adopted widely in universities, national laboratories, and research institutions for criticality calculations, shielding analysis, and reactor core modelling. The code is written in modern C++17 and provides a Python API for programmatic model construction and post-processing.

Despite its capabilities, OpenMC's official build system targets Linux and macOS exclusively. Windows users have historically been limited to two workarounds: using the Windows Subsystem for Linux (WSL2) or running a Docker container. Both approaches introduce non-trivial overhead — WSL2 requires enabling a virtualisation layer and navigating a Linux file system from within Windows, while Docker requires a container runtime and has performance implications for I/O-intensive simulations.

This technical note describes the installation of OpenMC natively on Windows using the openmc-windows-0.1-beta installer. This installs the OpenMC windows native executable into system and integrates with the OpenMC Python package installed via pip and requires no additional runtime environment beyond a set of redistributed DLLs.

2. Build Environment

2.1 Host System

Component	Details
Operating System	Windows 10 (64-bit)
C++ Compiler	GCC 15.2.0 (x86_64-w64-mingw32-g++)
Build System	CMake 3.x + GNU Make
Target Architecture	x86_64 (PE32+ Windows executable)
OpenMC Version	v0.15.3

3. Nuclear Data Library

OpenMC requires a cross-section data library in HDF5 format at runtime. The ENDF/B-VII.1 library in HDF5 format [3], provided by Argonne National Laboratory, is used in this distribution. The library path is communicated to OpenMC via the environment variable:

```
OPENMC_CROSS_SECTIONS=<path>\cross_sections.xml
```

The installer configures this variable automatically during setup.

4. Windows Installer

4.1 Installer Behaviour

The installation package (openmc-windows-v0.1-beta-installer.exe) performs the following actions automatically:

- Installs openmc.exe and all required runtime DLLs to the selected program directory
- Adds the installation directory to the Windows system PATH
- Prompts the user to select the folder containing cross_sections.xml
- Configures the OPENMC_CROSS_SECTIONS system environment variable
- Installs the openmc Python package via pip if Python 3.10+ is detected

4.2 System Requirements

Minimum System Requirements

- Windows 10 or Windows 11 (64-bit x86_64)
- Python 3.10 or newer (for the Python API)
- pip installed and available in PATH
- ~50 MB disk space for the executable and runtime libraries
- ~8 GB disk space for the ENDF/B-VII.1 HDF% nuclear data library

4.3 Environment Variable Configuration

The OPENMC_CROSS_SECTIONS environment variable is set at the system level during installation, so it is available in all subsequent Command Prompt and PowerShell sessions

without any manual configuration. A sign-out and sign-in cycle (or system restart) is required for the PATH and environment variable changes to take effect in open sessions.

5. Installation Verification

5.1 Executable

```
openmc --version
```

Expected output:

```
OpenMC version 0.15.3
Commit hash: 27e38e894697bb32a1dac7848d2618818b6b8daf
Copyright (c) 2011-2025 MIT, UChicago Argonne LLC, and contributors
Build type:          Release
Compiler ID:         GNU 15.2.0
MPI enabled:         no
PNG support:         yes
```

5.2 Python API

```
Python
import openmc
print(openmc.__version__)
```

Expected output: 0.15.3

5.3 Running a Simulation

Create a folder namely “OpenMC” where you have write permission. The Jezebel fast-fission criticality benchmark (included in the examples directory) provides a quick end-to-end test. Copy the example folder from c:/Program files (x86)/OpenMC/ to the “OpenMC” folder created.

```
cd D:\OpenMC\examples\jezebel
python jezebel.py
```

A successful run produces statepoint files and a summary of k-effective convergence.

6. Beta Version Limitations

Limitation	Detail
Particle count cap	Maximum 10,000 particles per batch in v0.1-beta
Batch limit	Maximum 200 batches per simulation in v0.1-beta
MPI support	Not enabled; single-node multithreading via OpenMP only

Full Version
The full version without particle/batch restrictions is available at:
https://shetsdp.github.io/blogs/openmc-windows.html

7. Known Issues and Troubleshooting

Symptom	Resolution
'openmc' is not recognized	Sign out and sign in; verify installation directory is in PATH
Missing DLL error on launch	Ensure all DLLs remain in the same directory as openmc.exe
OPENMC_CROSS_SECTIONS not set	Set manually via System Properties → Environment Variables
Python module not found	Run: python -m pip install openmc
HDF5 file open errors	Verify cross_sections.xml path contains no spaces

8. Conclusion

This technical note has documented a working procedure for compiling the OpenMC Monte Carlo code as a native Windows executable. The approach resolves the principal barrier to Windows adoption of OpenMC by eliminating the requirement for WSL2 or Docker, while retaining full compatibility with the OpenMC Python API and official HDF5 nuclear data libraries.

The beta release (v0.1-beta) imposes conservative limits on simulation size (10,000 particles, 200 batches) to facilitate testing and feedback. A full-capability release is planned and will be made available at the project page. Users and researchers are encouraged to report issues and contribute feedback to support ongoing development.

9. Important Links

OpenMC Windows Page: https://shetsdp.github.io/blogs/openmc-windows.html
GitHub Release Page:
Installation video:

10. References

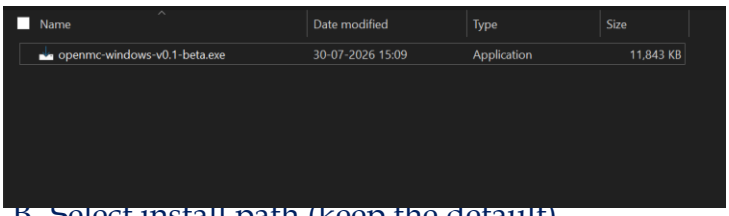
[1] P. K. Romano et al., "OpenMC: A State-of-the-Art Monte Carlo Code for Research and Development," Ann. Nucl. Energy 82 (2015) 90-97.

[2] OpenMC Documentation, https://docs.openmc.org

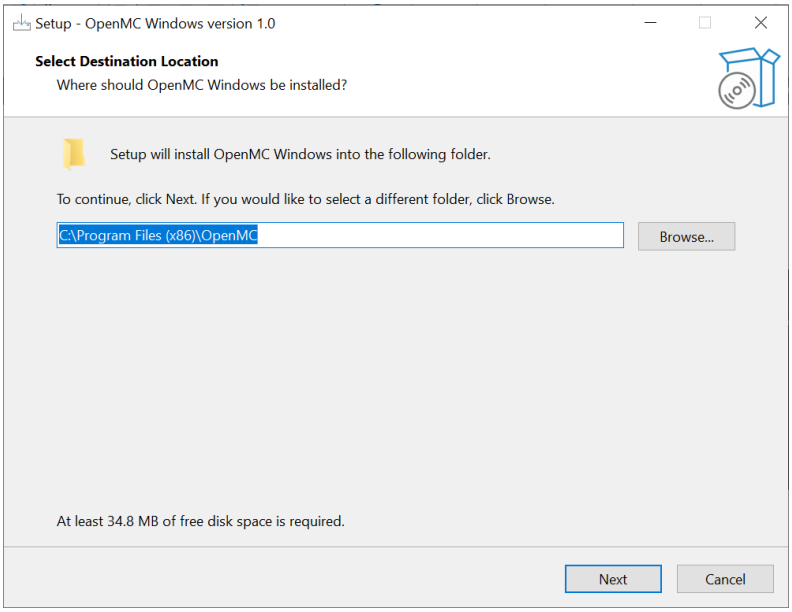
[3] ENDF/B-VII.1 Library (HDF5 format), Argonne National Laboratory, https://openmc.org/data/

11.Windows installation snapshots

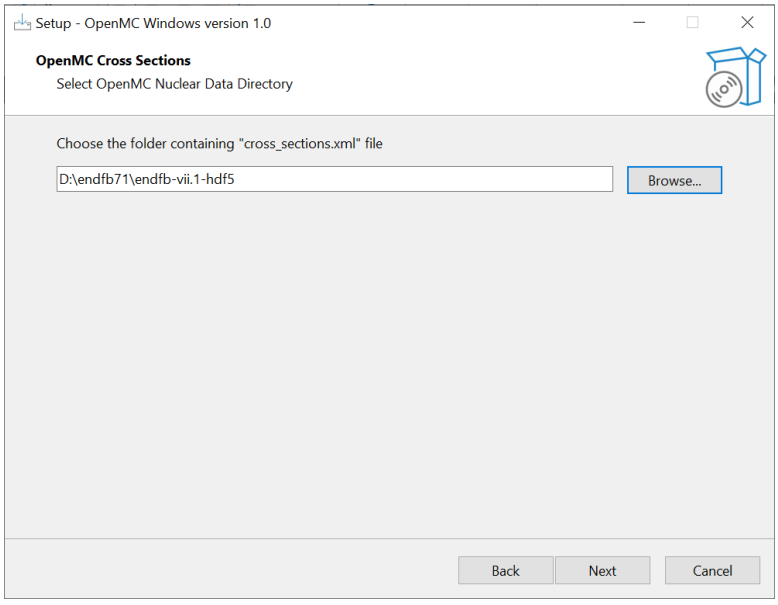
A. Execute



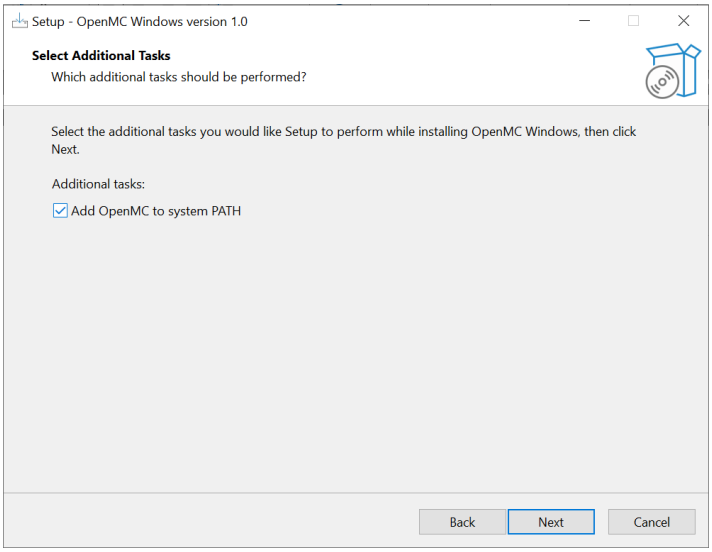
B. Select install path (keep the default)



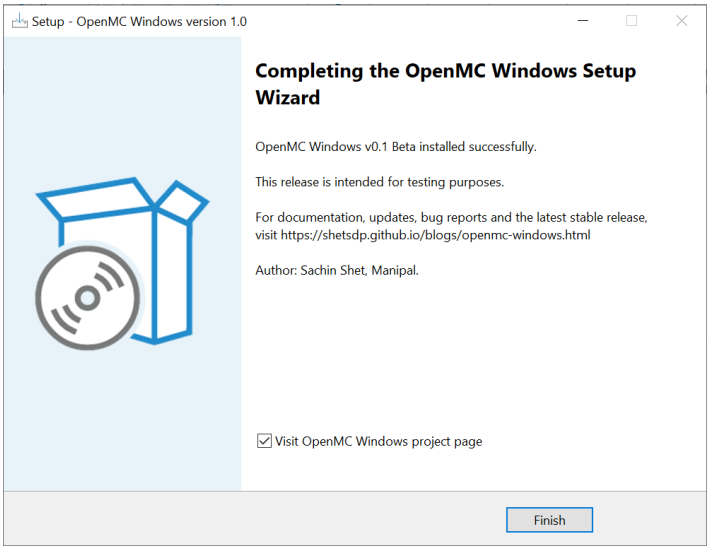
C. Choose HDF5 nuclear data directory (it must have cross_sections.xml)



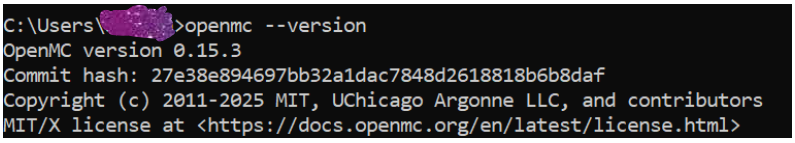
D. Check the box to add OpenMC to system PATH

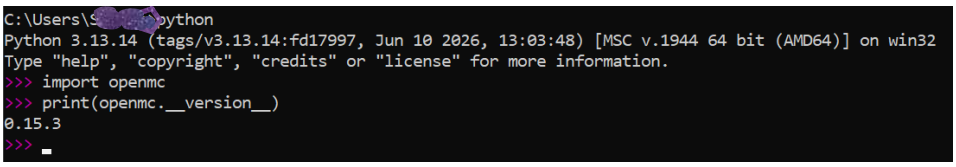


E. Click "Next" and Click "Install"

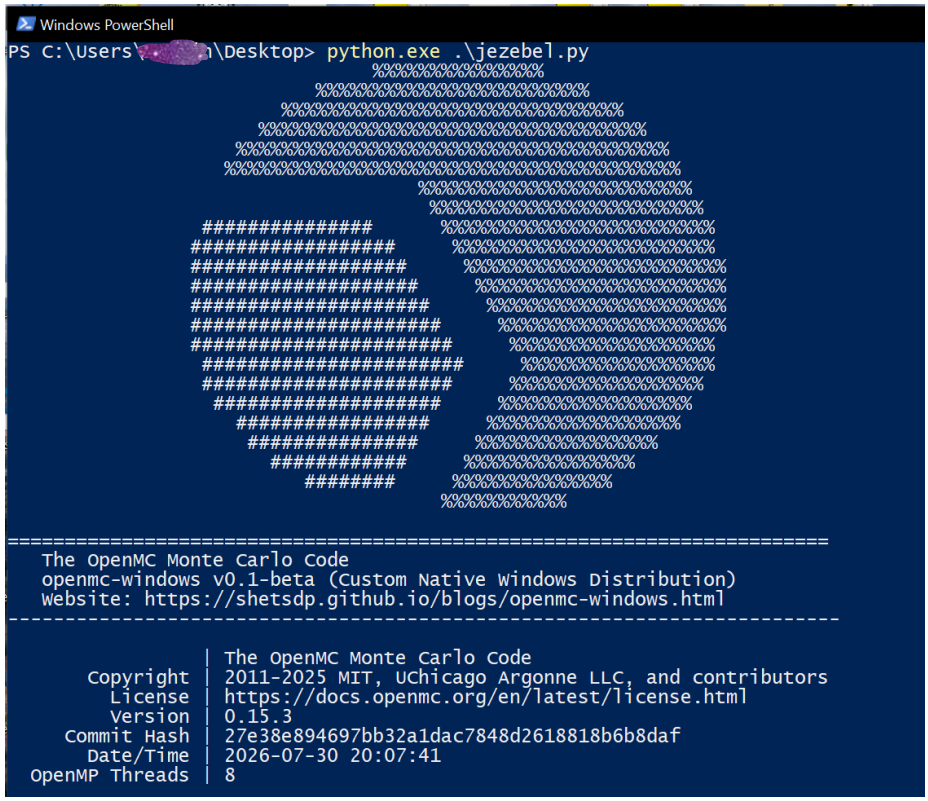


F. Verify Installation





G. Run an example (directory must have write permission, other than C:/)


Windows PowerShell
PS C:\Users\ \Desktop> python.exe .\jezebel.py
The OpenMC Monte Carlo Code
openmc-windows v0.1-beta (Custom Native Windows Distribution)
Website: https://shetsdp.github.io/blogs/openmc-windows.html
The OpenMC Monte Carlo Code
Copyright 2011-2025 MIT, UChicago Argonne LLC, and contributors
License https://docs.openmc.org/en/latest/license.html
Version 0.15.3
Commit Hash 27e38e894697bb32a1dac7848d2618818b6b8daf
Date/Time 2026-07-30 20:07:41
OpenMP Threads 8